Automatic generation of documentation

Code documentation software is a tool that has the ability to generate documentation from the source code of a program. This documentation allows other developers to understand the code easily. It ensures better maintainability of projects and is essential and highly recommended for group work.

These programs take into account the syntax and structure of the program language as well as the associated comments, provided that these are written in a suitable format. This workshop is an introduction to one of these tools, namely the Doxygen tool.

Doxygen :

Doxygen is an open source documentation generator capable of producing software documentation from the source code of a program. Doxygen is able to analyze source files written in C, Objective C, C#, PHP, C++, Java etc. In particular, Doxygen can generate documentation in HTML or LaTeX format from a set of source files. It can be used in two different ways:

- On the command line (case discussed in this workshop)
- Using the Doxygen wizard graphical interface

The official documentation for Doxygen can be found at this link Doxygen Manual: Overview.

In the following, we propose to document a mini SDL game "**MyGame**" which consists in scrolling the background. This project contains the following three files:

-Background.h

-Background.c

-Main.c

Background.h

```
#include <SDL/SDL.h>

#define CAMERA_W 100
#define CAMERA_H 200

struct background
{
 SDL_Rect bckg;
 SDL_Surface *img;
};
typedef struct background background;

void initBckg (background * b, char url[]); void

showBckg (SDL_Surface * screen, background b); void

scrollToLeft (background * b);

void scrollToRight (background * b);
```

Background.c

```
#include <SDL/SDL.h>
#include <SDL/SDL_image.h>
#include "background.h"

void initBckg (background * b, char url[]){
b-> img = IMG_Load (url);
 b-> bckg.x = 0;
 b-> bckg.y = 0;
 b-> bckg.w = b-> img->w;
 b-> bckg.h = b-> img-> h;
}

void showBckg (SDL_Surface * screen, background b){
SDL_BlitSurface (b.img, &(b.bckg), screen, NULL); }

void scrollToLeft (background * b){
 if (b-> bckg. x ! = 0)
 b-> bckg.x--;
}

void scrollToRight (background * b){
 if (b-> bckg.w - b-> bckg.x != CAMERA_W)
 b-> bckg.x++;
 }
```

Main.c

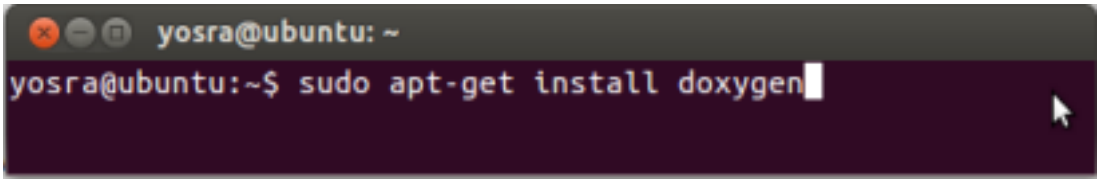
```
#include < stdlib.h>
#include < stdio.h>
#include <SDL/SDL.h>
#include <SDL/SDL_image.h>
#include "background.h"

int main (int argc, char *argv[]){
 background b;
```

```
 SDL_Event event;
 SDL_Surface *screen;
 if (SDL_Init (SDL_INIT_VIDEO) == -1) {
 printf ("Can't init SDL: %s\n", SDL_GetError ()); return
EXIT_FAILURE;
 }
 screen = SDL_SetVideoMode (CAMERA_W, CAMERA_H, 32,
SDL_HWSURFACE | SDL_DOUBLEBUF);
 initBckg (&b, "background.png");
 showBckg (screen, b);
 SDL_Flip (screen);
 int continue = 0;
 SDL_EnableKeyRepeat (SDL_DEFAULT_REPEAT_DELAY,
SDL_DEFAULT_REPEAT_INTERVAL);
 while (!continue) {
 while (SDL_PollEvent (&event)){
       switch (event.type) {
       box SDL_QUIT:
       continue = 1;
       break;
       box SDL_KEYDOWN:
       switch (event.key.keysym.sym)
             {
             box SDLK_LEFT:
              scrollToLeft (&b);
              break;
             box SDLK_RIGHT:
              scrollToRight (&b);
              break;
             }
       break;
       }

      }

 showBckg (screen, b);
 SDL_Flip (screen);
 }
 SDL_FreeSurface(screen);
 SDL_FreeSurface(b.img);
}
```

I. Installation :

The command to install the tool is as follows:

sudo apt-get install doxygen



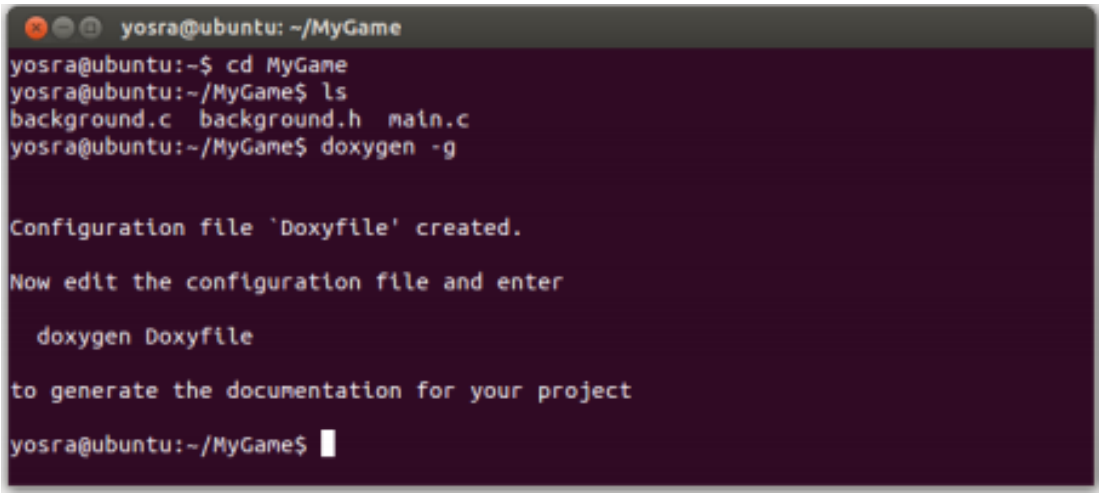
1-Installation of **Doxygen**

II. How to use Doxygen:

1. Generating a configuration file :

The first step is to generate a **Doxyfile** configuration file using the following command:

doxygen -g



2-Generation of Doxyfile

The **Doxyfile** configuration file is then edited with a text editor to change the main parameters:

- PROJECT_NAME: name of the project that groups the sources
- PROJECT_NUMBER: project version
- PROJECT_BRIEF: brief description of the project
- OUTPUT_DIRECTORY : destination directory, if this variable is left empty, the documentation will be generated in the current directory
- INPUT: source directories separated by spaces
- FILE_PATTERNS : list of source files to consider
- GENERATE_LATEX: indicates whether to generate documentation in LaTeX format ▪ OUTPUT_LANGUAGE: language used for the documentation (English, French, German, ...)
- OPTIMIZE_OUTPUT_FOR_C: Optimization of the documentation for the C language (YES,NO)

Example of a modified configuration file:

```
Ouvrir ▾   Doxyfile   ~/Bureau/My GAME/Code soure/src   Enregistrer
PROJECT_NAME           = "My GAME"

# The PROJECT_NUMBER tag can be used to enter a project or revision number. This
# could be handy for archiving the generated documentation or if some version
# control system is used.

PROJECT_NUMBER         = "Version 1"

# Using the PROJECT_BRIEF tag one can provide an optional one line description
# for a project that appears at the top of each page and should give viewer a
# quick idea about the purpose of the project. Keep the description short.

PROJECT_BRIEF          = "Ce projet est un jeu de plateforme utilisant la SDL1.2"

# With the PROJECT_LOGO tag one can specify a logo or an icon that is included
# in the documentation. The maximum height of the logo should not exceed 55
# pixels and the maximum width should not exceed 200 pixels. Doxygen will copy
# the logo to the output directory.

PROJECT_LOGO           =

# The OUTPUT_DIRECTORY tag is used to specify the (relative or absolute) path
# into which the generated documentation will be written. If a relative path is
# entered, it will be relative to the location where doxygen was started. If
# left blank the current directory will be used.

OUTPUT_DIRECTORY       = "/home/manel/Bureau/My GAME"
Texte brut ▾   Largeur des tabulations : 8 ▾   Lig 61, Col 54 ▾   INS
```

```
# Set the OPTIMIZE_OUTPUT_FOR_C tag to YES if your project consists of C sources
# only. Doxygen will then generate output that is more tailored for C. For
# instance, some of the names that are used will be different. The list of all
# members will be omitted, etc.
# The default value is: NO.

OPTIMIZE_OUTPUT_FOR_C  = YES
```

```
# If the GENERATE_LATEX tag is set to YES, doxygen will generate LaTeX output.
# The default value is: YES.

GENERATE_LATEX         = NO
```

3-Editing Doxyfile

2. Generation of documentation :

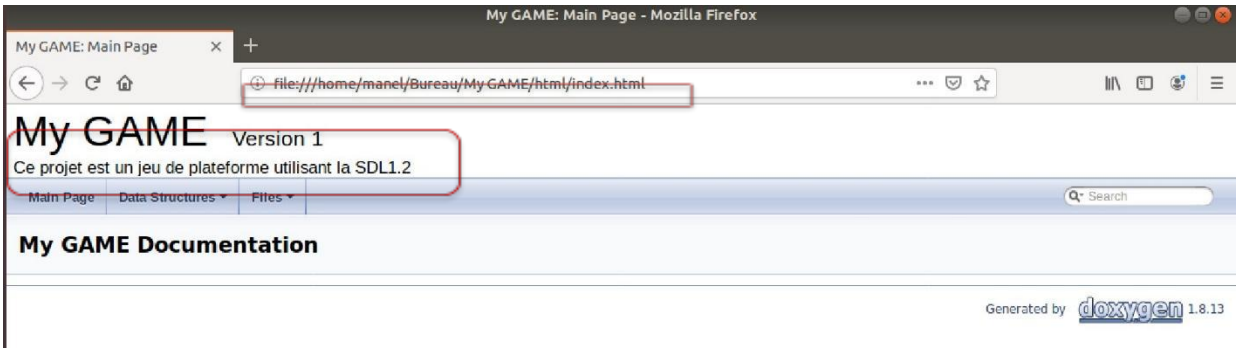
To generate the documentation, simply type the command: **doxygen Doxyfile**

```
yosra@ubuntu: ~
yosra@ubuntu:~$ doxygen Doxyfile
```

4-Command for generating documentation

An html directory is then generated which contains the code documentation. View the result by consulting the **index.html** file.

If you have chosen to generate LaTeX, a **latex** directory will also be created.



5-Generation of documentation

3. Setting up the documentation:

In this section, we will introduce you to some of the tags used for Doxygen.

3.1 Documentation blocks :

Various combinations are possible to create documentation blocks in C style.

```
/**
 * ... Documentation1 ...
 */

/* !
 * ... Documentation2 ...
 */
```

3.2 Header information :

The `@file [< name>]` tag is used to create a documentation block for a source or header file. To do this:

Further information can be added with the following tags:

@author: Allows you to add an author name

@brief: Allows you to create a short description in a documentation block **@date:** Allows you to add the date the file was created

@version: Add the current version number

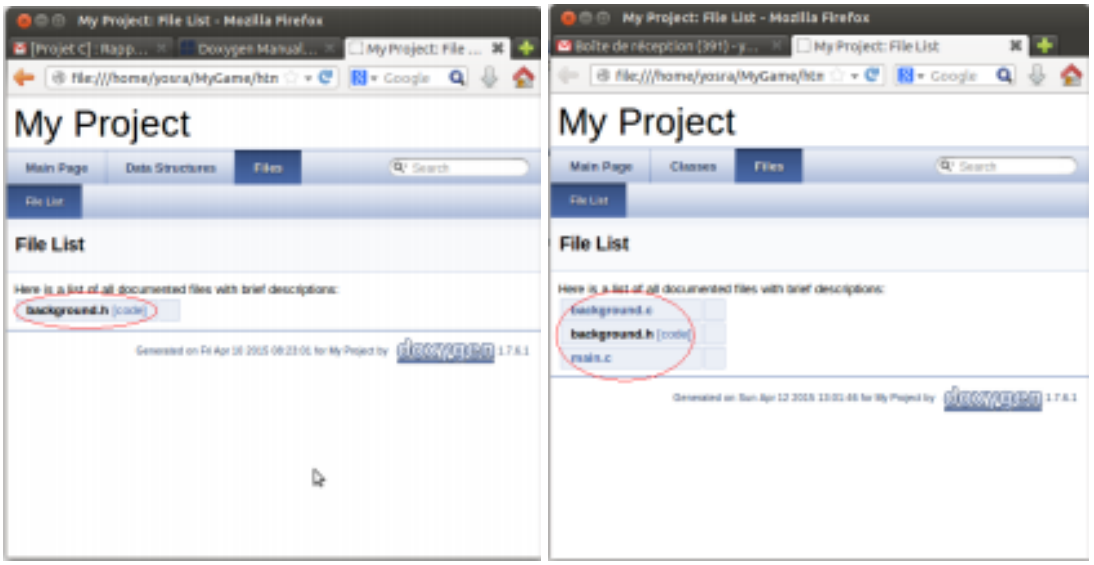
Example:

1. Add the documentation block to the main.c file header (and the background.c file)

```
/**
 * @file main.c
 * @brief Testing Program.
 * @author C Team
 * @version 0.1
 * @date Apr 01, 2015
 *
 * Testing program for background scrollilng
 *
 */
```

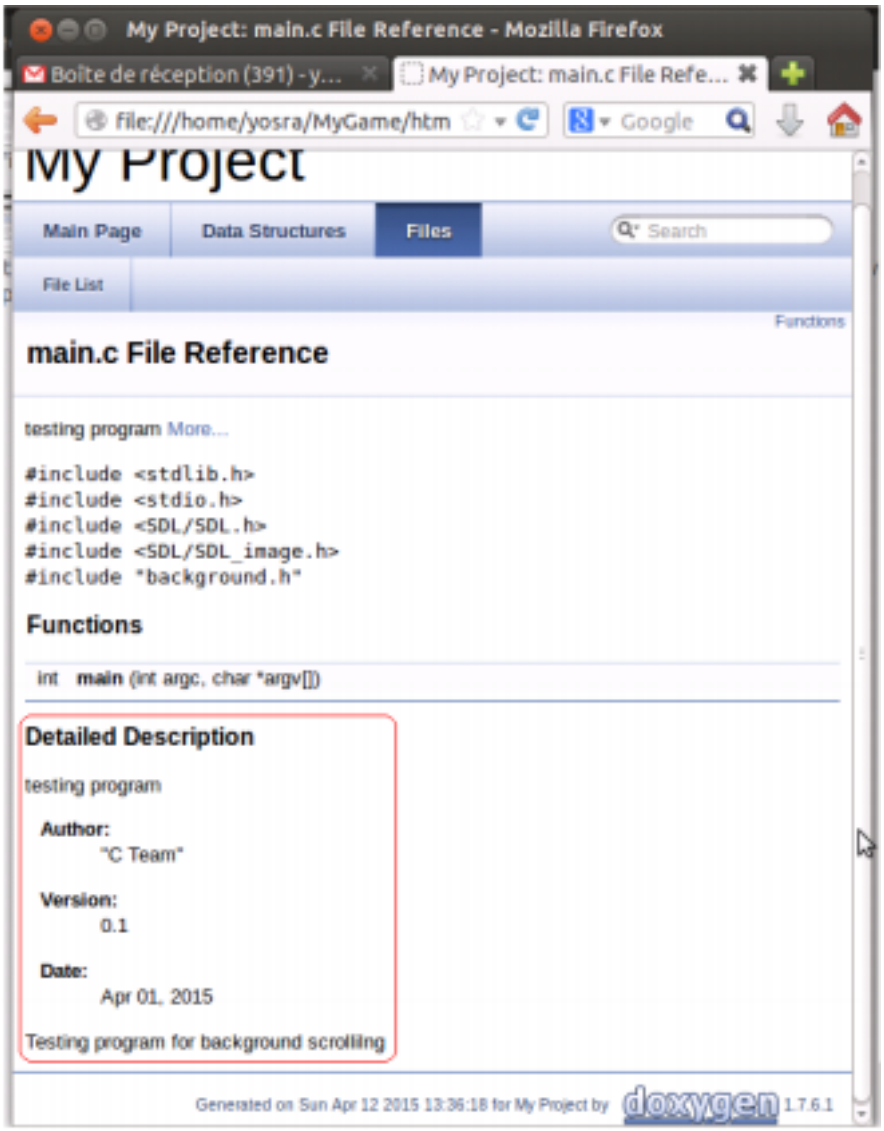
2. Retype command: **doxygen Doxyfile**

3. Go to the HTML folder and open the index.html file with your browser.
4. Click on the "Files" tab and then "File List". You can see that the two files are added to the list.



6-Before the@file tag 7-After the@file tag

5. Click on the "Files" tab and then on "main.c".



8-Adding information from the top

3.3 Documentation of a structure :

This is the "@struct" tag to be placed just before the structure.

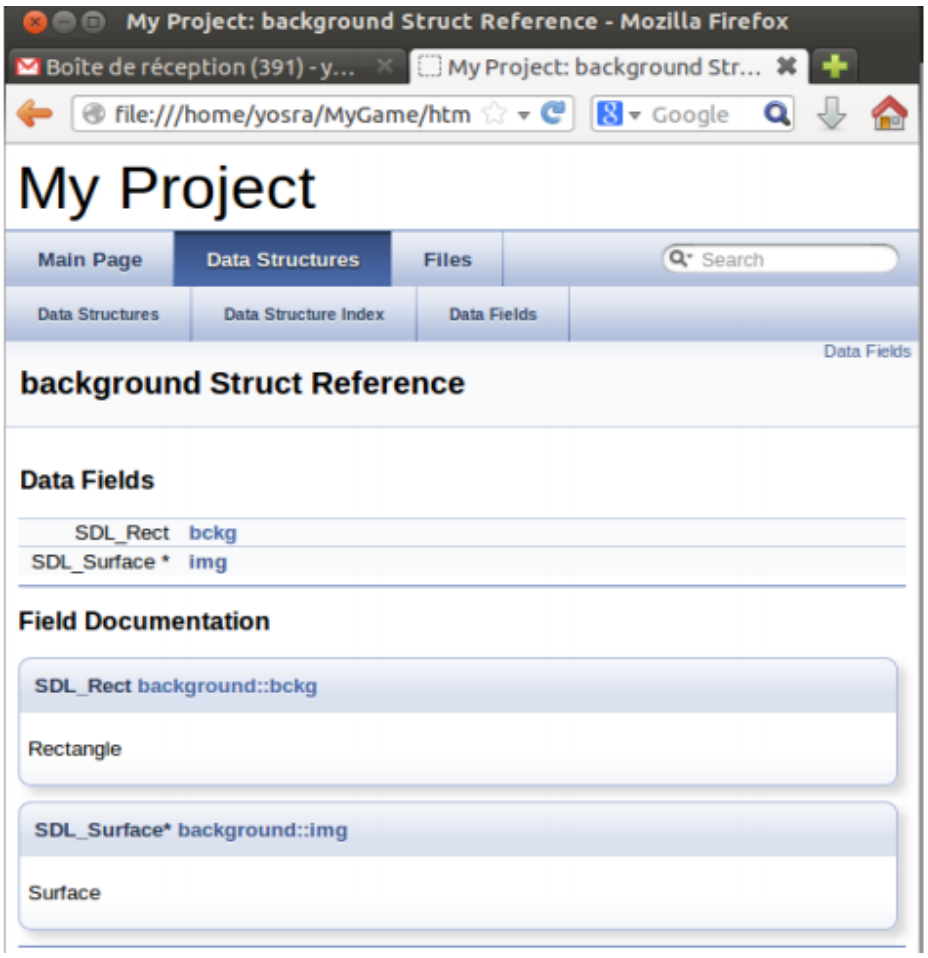
Example:

```
/**
 * @struct Background
 * @brief struct for background
*/
struct background
{
  SDL_Rect bckg; /*!< Rectangle*/
 SDL_Surface *img; /*!< Surface. */
 }
typedef struct background background;
```

The "<" operator is used to document a member, in this case of a

structure. Result: Click on the "Data Structures" tab.

Note that after each modification the documentation generation command must be retyped: **doxygen Doxyfile**



9-Documentation of a structure

3.4 Documentation of a function :

For tags that allow a function to be documented, we cite among others

@param: Used to describe the parameters of a function

@ return: Used to describe the return of the function.

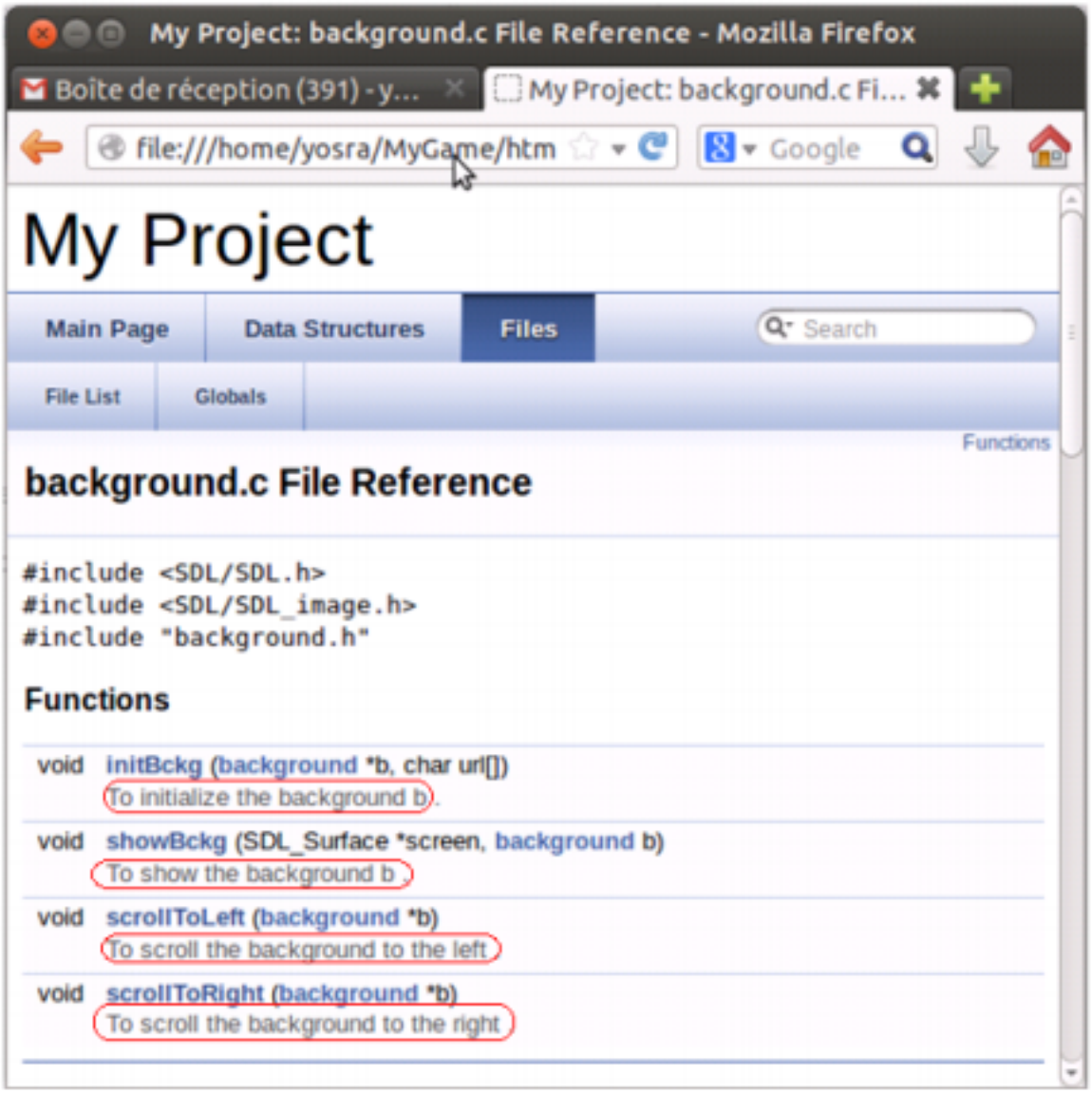
Example: we have documented the functions in the file background.c :

```
        /**
 * @file background.c
*/
#include <SDL/SDL.h>
#include <SDL/SDL_image.h>
#include "background.h"
/**
 * @brief To initialize the background b .
```

```
 * @param b the background
 * @param url the url of the image
 * @return Nothing
 */
void initBckg (background * b, char url[]){
 b-> img = IMG_Load (url);
 b-> bckg.x = 0;
 b-> bckg.y = 0;
 b-> bckg.w = b-> img->w;
 b-> bckg.h = b-> img-> h;
}
/**
 * @brief To show the background b .
 * @param scren the screen
 * @param b the background
 * @return Nothing
 */
void showBckg (SDL_Surface * screen, background b){
 SDL_BlitSurface (b.img, &(b.bckg), screen, NULL);
}
/**
 * @brief To scroll the background to the left .
 * @param b the background
 * @return Nothing
 */
void scrollToLeft (background * b){
 if (b-> bckg. x ! = 0)
 { b-> bckg.x--; }
}
/**
 * @brief To scroll the background to the right .
 * @param b the background
 * @return Nothing
 */
void scrollToRight (background * b){
 if (b-> bckg.w - b-> bckg.x != CAMERA_W)

 b-> bckg.x++;
}
```

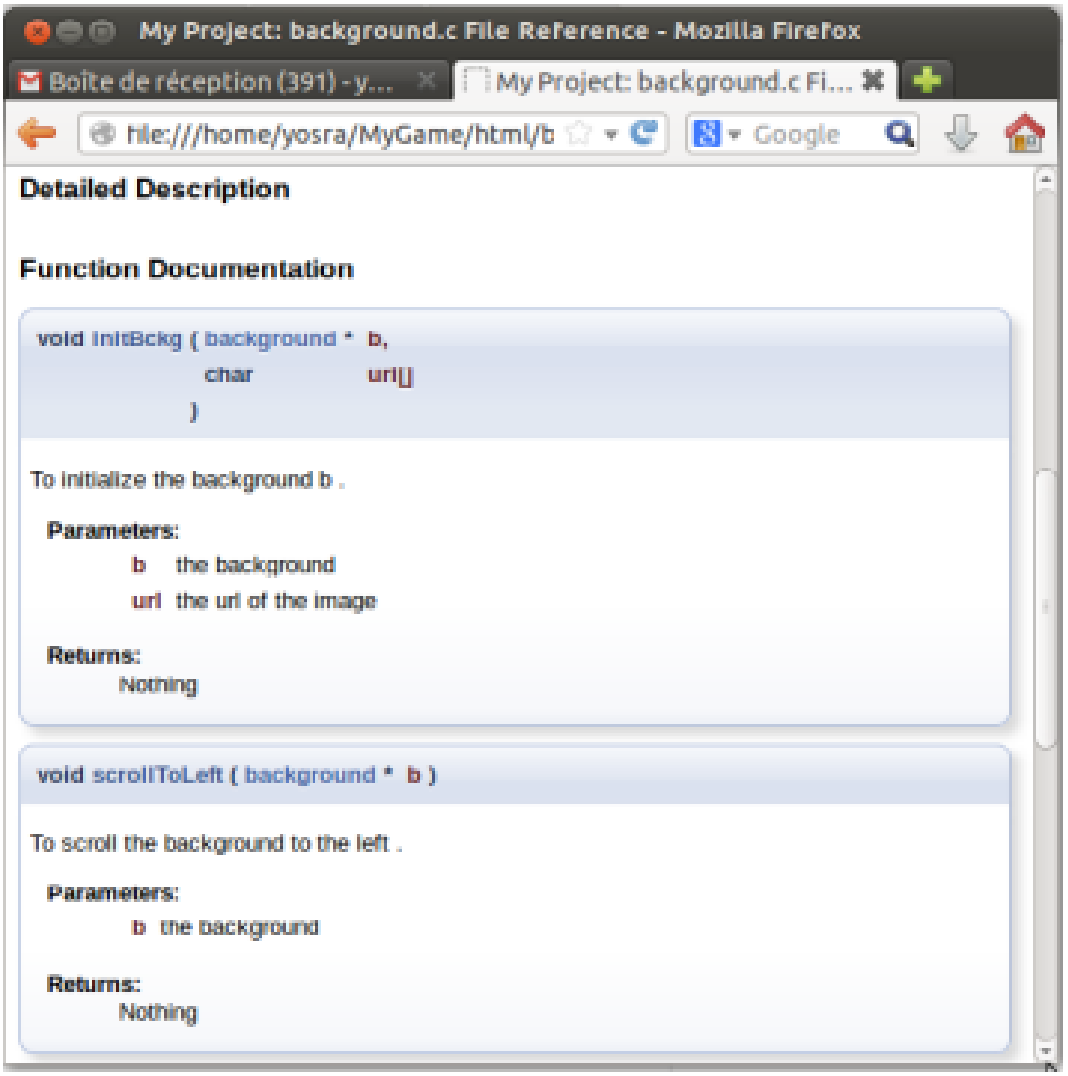
Result:

In this figure we see that the short description of each function is added under the headers:



10-Short description

Here you can see the detailed description:



11-Detailed description

Conclusion:

This workshop is an introduction to the automatic generation of documentation using the Doxygen tool. However, you should be aware that there are other tags that can be used for documentation. For the complete list, go to the official website: Doxygen Manual: Special Commands